

TECHNICAL PRESENTATION

BLOCK AVOID 3D

左右分裂・合体ギミックによる直感的な3D障害物回避ゲーム

横浜デジタルアート専門学校 ゲーム科3年 / 田上 ヒデオ

01. 作品概要と仕様

Core Concept & System Specs

直感操作の **3Dアクション**

プレイヤーをスワイプ操作で左右へダイナミックに分裂・合体させながら、迫り来る障害物を回避してゴールを目指すハイスピード3Dアクションゲームです。スマートフォンでも直感的に操作できるよう、スワイプ操作の分かりやすさとテンポの良いゲームプレイを意識しました。

ジャンル	3D障害物回避ゲーム
動作環境	PC / iOS / Android
開発環境	Unity / C#
開発期間	2026年3月～



アピールポイントとこだわり



直感制御

スマートフォンでの操作性を高めるため、スワイプ方向に応じて、キャラクターの分裂・合体が自然に反応するようにし、ストレスの少ない操作体験を目指しました



破綻のない管理

分裂数管理用の変数 cloneCnt と状態フラグを厳密に調和。激しい連続入力下でも同期ズレを起こさず、不整合が起きにくい設計にしました。



レスポンス最適化

当たり判定の範囲や移動タイミングを調整し、見た目と接触判定のズレを減らすことで、プレイヤーが納得しやすい操作感を目指しました。

02. システムアーキテクチャ

Game System & Architecture Design

ゲーム マネジメント設計

GameManager

ゲーム状態と進行データの一元管理

現在の状態（プレイ中・ゲームオーバー・クリア）をプロパティ管理し、他クラスから参照可能に。リトライ時には、ゲーム状態フラグやステージ進行に関わる値を初期化します。

PlayerManager

プレイヤークラスの統括・挙動制御

プレイヤー関連処理の中心となるクラス。コルーチンを利用した滑らかな分裂・合体のアニメーションおよび分裂数のコントロールを行います。障害物との接触をトリガーで検知し、状態遷移を通知します。

状態遷移とゲームフロー

2. GAMEPLAY

移動制御、スワイプ判定。リアルタイムでのプレイヤーオブジェクトの分裂・合体。

4. RETRY & RESET

UIManagerから各マネージャーのリセット処理を呼び出し、ゲーム状態をリセットし、すぐに再プレイできる状態に戻します

1. TITLE

初期化。プレイヤーが「STAGE」または「ENDLESS」を選択してゲーム画面へ遷移。

3. JUDGMENT

障害物接触によるゲームオーバー、または最終ゴール到達でのゲームクリア判定

。

分裂・合体 システムのロジック

1. 状態管理 (cloneCnt)

プレイヤーの最大数を3体に制限 (cloneCnt : 0~2)

(0 : 1体表示 / 1 : 2体表示 / 2 : 3体表示)

cloneCntの値に応じて表示するプレイヤーオブジェクトを切り替え、それぞれの配置座標を決定します。

2. 表示/非表示の切り替え

単一オブジェクトの変形ではなく、事前に生成されたプレハブ群の表示切り替えを制御。移動開始直前と完了後の適切なタイミングで差し替え、自然な視覚表現を演出します。

滑らかなコア移動ロジック

瞬間移動から補間へ

UnityのVector3.MoveTowardsとyield return nullをコルーチンで制御。指定された目的地（movepos）に到達するまでフレーム単位で補間処理します。不自然な飛び跳ねを排除し、操作時の違和感を減らしました。

```
// 目的地に近づくまで移動を継続  
Distance ( pos , movepos ) > 0.01f
```

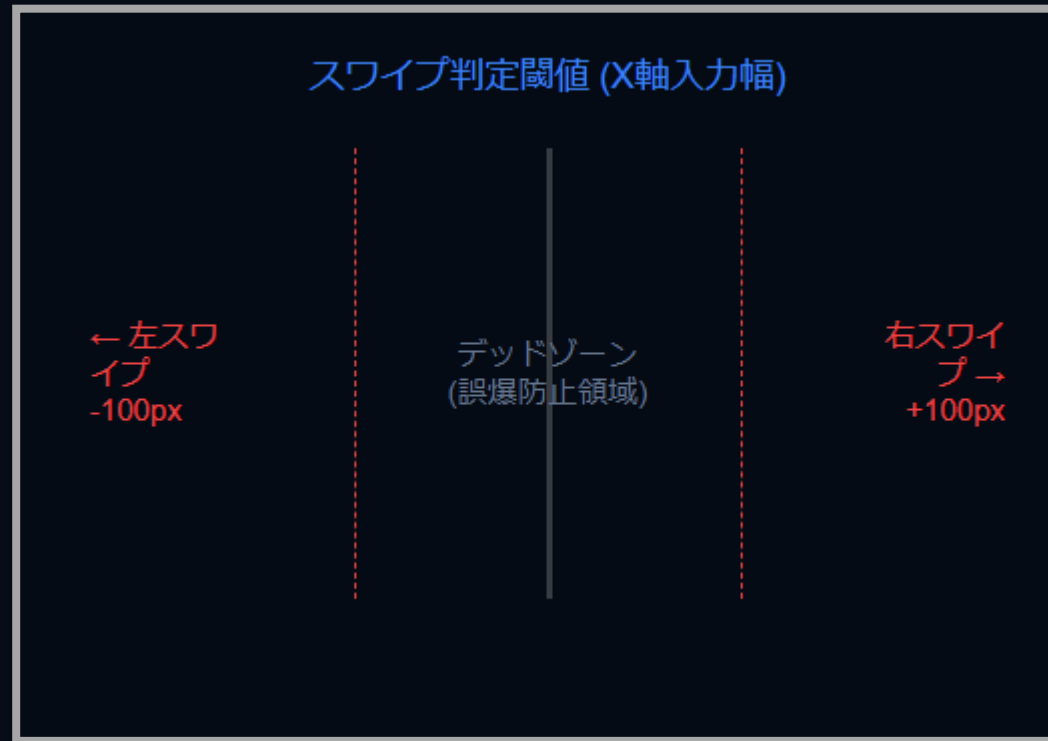
```
// 移動コルーチン  
while (Vector3.Distance(pos, movepos) > 0.01f) {  
    pos = Vector3.MoveTowards(  
        pos,  
        movepos,  
        speed * Time.deltaTime);  
    yield return null;  
}  
pos = movepos; // 座標を完全同期
```

タッチ・スワイプの判定基準

デバイス依存を排除した設計

スマートフォンの画面サイズに依存せず安定した判定を行うため、タッチ開始地点と現在地点のX座標差 (Δx) を常時測定します。閾値を100pxのデッドゾーンに設定することで、画面タッチの微小なブレを判定しないようにしています。

- $\Delta x < -100\text{px}$: 左スワイプ (合体)
- $\Delta x > 100\text{px}$: 右スワイプ (分裂)



チュートリアルシステムの設計

「非侵入型」設計

ゲーム体験を阻害しない仕組み

没入感を維持するため、本編のコアロジックに大きく手を加えず、独立した TutorialManager として開発。状態監視と介入条件の明確な分離を確立しました。

高拡張コンポーネント

疎結合による高いメンテナンス性

TutorialHeart.cs や TutorialObstacle.cs をそれぞれ個別のクラスとして実装。既存のメインロジックへの影響を抑えられるため、安全に将来のアップデートや変更に対応可能です。

衝突・クリア判定フロー

クラス名 / 役割	イベントトリガー & 処理内容	設計の意図・疎結合アプローチ
Player.cs 物理検知	OnTriggerEnter で障害物との接触を検知。即座に PlayerManager へ衝突をコール。	衝突の物理検知のみに専念。ゲームオーバーの実ロジックは自身で持たない。
PlayerManager.cs オブジェクト統括	衝突通知を受け、GameManager へ「ゲームオーバー状態への移行」を依頼。	プレイヤーの状態（生死）を管理し、システムに伝える仲介役に徹する。
GameManager.cs 進行制御	状態フラグ gameOverFlg を true に更新。進行を一時凍結。	ゲームシステム全般を制御。直接UIを操作せず、純粋な状態遷移のみを担当。
UIManager.cs UI表示制御	gameOverFlg（ゲームオーバー状態フラグ）の変更を毎フレーム監視し、検知時にゲームオーバー画面を描画	UI表示ロジックをゲームの内部ロジックと切り分け、保守性を向上。

技術的課題と リファクタリング成果



リトライ時の状態不整合の改善

初期プロトタイプで発生した「リトライ時に分裂カウントが同期ズレを起こし進行不能になる不具合」を、UIManagerから順序よく安全にマネージャー群をリセットする再設計によって根本解決しました。



疎結合を意識したコード設計

マネージャー、プレイヤー、UIスクリプト間の相互依存を減らすように設計。状態監視フラグとイベント通知システムを仲介させることで、可読性・再利用性の向上を図りました。



今後の仕様追加を見据えた拡張性の確保

コルーチンを中核にした柔軟な移動制御により、今後の仕様変更（4体以上への分裂）や、特殊ボーナスアイテムの実装にもロジックを破壊することなく容易に対応可能です。

ご覧いただき
ありがとうございました

横浜デジタルアーツ専門学校 ゲーム科3年 / 田上 ヒデオ